

Conectores de Base de datos

Gregurak: Manual de conector de BD (Explicacion del tema, explicación paso a paso sobre como insertar los conectores en la BD con imagenes en lo posible, link del video explicativo y links de programas utilizados en el proceso)

Nahir: Explicacion (Explicacion sobre los conectores de bases de datos, como funcionan, para que funcionan y como podemos insertarlos en una BD)

Uriel: Video del paso a paso de como insertar el conector en la BD, explicacion de como funcionan los conectores y explicar que hizo cada participante en el trabajo (Debe durar 3 minutos como mínimo a 7 minutos como mucho).

Conectores de Bases de Datos

¿Qué es un conector?

El "conector" es SQLite compilado a WebAssembly (WASM) que se ejecuta dentro del navegador.

El más usado se llama sql.js. Es literalmente el motor de SQLite (escrito en C) compilado a un archivo .wasm que el navegador descarga y ejecuta como JavaScript. No hay servidor, no hay puerto, no hay localhost. Todo ocurre en la pestaña del usuario.

¿Cómo funciona?

1. El navegador descarga sql-wasm.js y sql-wasm.wasm desde un CDN.
2. initSqlJs() carga el motor SQLite en memoria.
3. new SQL.Database() crea una base de datos en RAM (vacía o desde un archivo .sqlite cargado).
4. Ejecutas SQL normal (CREATE, INSERT, SELECT...) con db.run() o db.exec().
5. Si quieres persistir, exportas la BD como bytes y la guardas en localStorage o IndexedDB.

¿Para qué sirve?

- Prototipos y demos: probar SQL sin montar un servidor.
- Apps offline / PWA: funcionan sin internet.
- Educación: enseñar SQL en el navegador.
- Un solo usuario: cada visitante tiene su propia BD local.

Limitaciones:

-  No sirve para multiusuario ni aplicaciones web reales con datos compartidos.
-  Los datos se pierden si no los persistes manualmente.
-  No hay seguridad, autenticación ni concurrencia.

¿Cómo insertar en una BD?

```
html
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <script src="https://cdnjs.cloudflare.com/ajax/libs/sql.js/1.8.0/sql-wasm.js"></script>
</head>
```

```

<body>
  <input type="text" id="nombre" placeholder="Nombre">
  <input type="text" id="email" placeholder="Email">
  <button onclick="insertar()">Guardar</button>

  <script>
    let db;

    // 1. Inicializar el "conector" (equivalente a cargar el driver JDBC)
    initSqlJs({
      locateFile: file => `https://cdn.jsdelivr.net/npm/sql.js/dist/sql.js`
    }).then(function(SQL) {
      // 2. Recuperar BD guardada o crear una nueva
      const guardado = localStorage.getItem("mi_bd");
      db = guardado
        ? new SQL.Database(new Uint8Array(JSON.parse(guardado)))
        : new SQL.Database();

      // 3. Crear tabla si no existe
      db.run(`CREATE TABLE IF NOT EXISTS usuarios (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        nombre TEXT,
        email TEXT
      )`);
    });

    // 4. Función para insertar
    function insertar() {
      const nombre = document.getElementById("nombre").value;
      const email = document.getElementById("email").value;

      // Consulta parametrizada (evita inyección SQL)
      db.run(
        "INSERT INTO usuarios (nombre, email) VALUES (?, ?)",
        [nombre, email]
      );

      // Persistir en localStorage
      localStorage.setItem("mi_bd", JSON.stringify(Array.from(db.export())));

      alert("Registro insertado correctamente");
    }
  </script>
</body>
</html>

```

=====

Manual Paso a Paso: Integración de Base de Datos SQL con "Node.js" y "JavaScript".
Este manual describe el procedimiento detallado para instalar, configurar y desarrollar una aplicación web con Node.js, Express y SQLite (better-sqlite3), permitiendo insertar y consultar información mediante un formulario web.

1.Requisitos y preparación del entorno.

[Node.js](#) Windows 10

[Node.js](#) Linux

Antes de comenzar, asegúrate de tener instalado:

- Node.js: Puedes verificar la instalación en la terminal con "node -v"
- Tocá acá para descargar la versión de Windows.
- Un editor de código (como Visual Studio Code o IntelliJ).

Pasos de inicialización:

- Abre la terminal o consola de comandos (CMD).
- Crea un directorio de trabajo y navega hasta él:

"mkdir proyecto-sqlite-node" (esto creará una carpeta)

"cd proyecto-sqlite-node" (y con "cd" entrará a la carpeta)

- Inicializa el proyecto Node.js para generar el archivo package.json: "npm init -y"

- Instala las librerías necesarias:

- Express: Para la creación de la API y servidor HTTP:

"npm install express"

- Better-SQLite3: El conector nativo y ligero de SQLite para Node.js:

"npm install better-sqlite3"

2 Creación del Servidor y Configuración del Conector SQL.

Crea un archivo llamado "[server.js](#)" en la raíz del proyecto. En este archivo se realiza la inicialización de la base de datos, la creación de la estructura de tablas y el mapeo de rutas.

3 Interfaz de Usuario y Lógica Cliente.

Crea una carpeta llamada "public" en el directorio principal y dentro de ella crea un archivo "[index.html](#)".

4 Estructura Final del Proyecto

La jerarquía de archivos debe quedar de la siguiente manera:

```
proyecto-sqlite-node/  
├── node_modules/  
├── public/  
│   └── index.html  
└── package.json
```

- package-lock.json
- server.js
- sistema.db (se crea automáticamente al iniciar)

5. Verificación y Pruebas

1. Iniciar el Servidor.

Abre la terminal en la raíz de la carpeta del proyecto y ejecuta:
"node server.js"

Verás el mensaje: Servidor activo en "http://localhost:3000"

2. Acceder a la Aplicación Web.

Abre el navegador web de tu preferencia e ingresa a la dirección: "http://localhost:3000"

3. Probar Inserción de Datos (INSERT)

Ingresa un Nombre y un Correo en los campos del formulario y presiona Guardar Registro. Verifica que el servidor procesa la petición y se crea el archivo sistema.db automáticamente en caso de no existir.

4. Probar Extracción de Datos (SELECT)

Comprueba que el elemento de la tabla se actualiza de manera inmediata sin recargar la página, haciendo la llamada fetch al endpoint GET.

=====

```

const express = require('express');
const Database = require('better-sqlite3');
const path = require('path');

const app = express();
const PORT = 3000;

app.use(express.json());
app.use(express.urlencoded({ extended: true }));
app.use(express.static(path.join(__dirname, 'public')));

const db = new Database('sistema.db');

db.exec(`
  CREATE TABLE IF NOT EXISTS usuarios (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre TEXT NOT NULL,
    email TEXT NOT NULL
  )
`);

app.post('/api/usuarios', (req, res) => {
  const { nombre, email } = req.body;

  if (!nombre || !email) {
    return res.status(400).json({ error: 'El nombre y el email son obligatorios.' });
  }

  const stmt = db.prepare('INSERT INTO usuarios (nombre, email) VALUES (?, ?)');
  const result = stmt.run(nombre, email);

  res.json({ id: result.lastInsertRowid, nombre, email });
});

app.get('/api/usuarios', (req, res) => {
  const stmt = db.prepare('SELECT * FROM usuarios ORDER BY id DESC');
  const usuarios = stmt.all();

  res.json(usuarios);
});

app.listen(PORT, () => {
  console.log(`Servidor activo en http://localhost:${PORT}`);
});

```

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Integración SQL con Node.js y Express</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      max-width: 600px;
      margin: 40px auto;
      padding: 20px;
      background-color: #f4f4f9;
    }
    h1, h2 {
      color: #333;
    }
    form {
      display: flex;
      flex-direction: column;
      gap: 10px;
      margin-bottom: 30px;
      background: #fff;
      padding: 20px;
      border-radius: 8px;
      box-shadow: 0 2px 4px rgba(0,0,0,0.1);
    }
    input, button {
      padding: 10px;
      font-size: 16px;
      border: 1px solid #ccc;
      border-radius: 4px;
    }
    button {
      background-color: #007bff;
      color: white;
      border: none;
      cursor: pointer;
    }
    button:hover {
      background-color: #0056b3;
    }
    table {
      width: 100%;
      border-collapse: collapse;
      background: #fff;
      box-shadow: 0 2px 4px rgba(0,0,0,0.1);
    }
  </style>
</head>
<body>
  <h1>Integración SQL con Node.js y Express</h1>
  <div>
    <form>
      <input type="text" value="Nombre de usuario:" />
      <input type="password" value="Contraseña:" />
      <button type="submit" value="Iniciar Sesión" />
    </form>
    <table>
      <thead>
        <tr>
          <th>ID</th>
          <th>Nombre</th>
          <th>Apellido</th>
          <th>Email</th>
          <th>Fecha de Registro</th>
        </tr>
      </thead>
      <tbody>
        <tr>
          <td>1</td>
          <td>Juan</td>
          <td>Pérez</td>
          <td>juan.perez@example.com</td>
          <td>2023-10-27</td>
        </tr>
        <tr>
          <td>2</td>
          <td>María</td>
          <td>García</td>
          <td>maria.garcia@example.com</td>
          <td>2023-10-28</td>
        </tr>
        <tr>
          <td>3</td>
          <td>Carlos</td>
          <td>Rodríguez</td>
          <td>carlos.rodriguez@example.com</td>
          <td>2023-10-29</td>
        </tr>
        <tr>
          <td>4</td>
          <td>Ana</td>
          <td>López</td>
          <td>ana.lopez@example.com</td>
          <td>2023-10-30</td>
        </tr>
        <tr>
          <td>5</td>
          <td>Pedro</td>
          <td>Martínez</td>
          <td>pedro.martinez@example.com</td>
          <td>2023-10-31</td>
        </tr>
      </tbody>
    </table>
  </div>
</body>
</html>
```

```

th, td {
  border: 1px solid #ddd;
  padding: 10px;
  text-align: left;
}
th {
  background-color: #007bff;
  color: white;
}
</style>
</head>
<body>

<h1>Registro de Usuarios</h1>

<form id="formUsuario">
  <label for="nombre">Nombre:</label>
  <input type="text" id="nombre" name="nombre" required placeholder="Ej: Juan Pérez">

  <label for="email">Correo Electrónico:</label>
  <input type="email" id="email" name="email" required placeholder="Ej:
juan@example.com">

  <button type="submit">Guardar Registro</button>
</form>

<h2>Usuarios Registrados</h2>
<table>
  <thead>
    <tr>
      <th>ID</th>
      <th>Nombre</th>
      <th>Email</th>
    </tr>
  </thead>
  <tbody id="tablaUsuarios">
    <!-- Los registros se cargan dinámicamente -->
  </tbody>
</table>

<script>
const form = document.getElementById('formUsuario');
const tabla = document.getElementById('tablaUsuarios');

async function cargarUsuarios() {
  const res = await fetch('/api/usuarios');
  const usuarios = await res.json();

```



```

    tabla.innerHTML = "";
    usuarios.forEach(u => {
        const fila = document.createElement('tr');
        fila.innerHTML = `
            <td>${u.id}</td>
            <td>${u.nombre}</td>
            <td>${u.email}</td>
        `;
        tabla.appendChild(fila);
    });
}

    form.addEventListener('submit', async (e) => {
        e.preventDefault();

        const nombre = document.getElementById('nombre').value;
        const email = document.getElementById('email').value;

        await fetch('/api/usuarios', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ nombre, email })
        });

        form.reset();
        cargarUsuarios(); // Actualiza la tabla dinámicamente
    });

    cargarUsuarios();
</script>

</body>
</html>

```